Runtime State Verification on Resource-Constrained Platforms

John Clemens
Johns Hopkins APL

Raj Pal
Department of Defense

Branden Sherrell
Johns Hopkins APL

***Abstract*—Verifying the integrity of multi-component cyber and cyber-physical systems remains a crucial and necessary precondition for trust. The edge components of such systems, those that interact directly with the user or physical world, are typically resource-constrained – with minimal CPU, memory, and power – and do not support traditional mechanisms for trustworthy integrity verification. Yet the integrity of *all* components of a system, including resource-constrained edge devices, is essential to guard against compromise and to ensure user confidence. In this paper we present *IoTA*, an extensible, open source, agent-based framework for trustworthy runtime integrity verification on resource-constrained platforms. We define IoTA's system architecture, compare it with existing integrity measurement frameworks, and discuss the necessary trade-offs to achieve trust within these constraints. Further, we present a prototype implementation of IoTA to explore both the feasibility and impact of agent-based measurement.**

I. Introduction

Runtime integrity verification is a critical component for evaluating the trustworthiness of computing platforms. The process involves collecting sufficient platform evidence to evaluate the system state *at the moment the evidence is collected*. This is in contrast with boot-time state verification, which simply guarantees that the system was in a known-good state *at the moment when the system started*. Since systems change state after boot via unpredictable interactions with their environment, both runtime and boot-time evidence are needed to evaluate integrity. Collecting and evaluating integrity evidence is referred to as *measurement and attestation* (M&A).

Resource-constrained systems are becoming more important as multi-component systems incorporate platforms with limited capabilities. Systems containing resource-constrained devices support a wide variety of applications, including the Internet of Things (IoT), industrial automation, the smart grid, and smart cities. Such broad adoption in highly visible parts of society and infrastructure highlights the need for verifiable trust within these systems.

M&A remains an active area of research for all computing platforms, but is particularly problematic on resource-constrained systems. Direct runtime integrity evidence is gathered from an agent located on the system under evaluation. Indirect evidence, such as observed system behavior, can also inform an evaluation, but is not a primary focus of this research. However, resource-constrained devices lack the resources necessary to support a traditional runtime collection agent longside the main application.

In this paper we present IoTA, an extensible, open, agent-based framework for the trustworthy collection and evaluation of runtime integrity evidence from resource-constrained platforms. IoTA is designed to leverage modern microcontroller hardware capabilities to provide similar integrity guarantees as verification agents for traditional platforms, while remaining small enough to co-exist with the main application. The main contributions of this paper are a novel architecture for trustworthy measurement of resource-constrained devices and an analysis of a prototype implementation of that architecture.

The remainder of this paper is organized as follows. Section II provides some background and motivation for IoTA and enumerates challenges and related work in the field. Section III provides an overview of IoTA's architecture and explains both how it provides the necessary properties and what trade-offs are necessary. Section IV details our prototype, IoTA, including real-world overhead metrics. And Section V summarizes IoTA and discusses future areas for improvement.

II. Background

Much has been written about the current state of IoT device security and the potential harm that can come from untrustworthy use of those systems (see e.g. [1]). Examples include lack of integrity on point of sale systems allowing malware to steal credit card information [2], resulting in financial harm to both corporations and consumers alike; the massive botnet created by exploiting default credentials on Internet-connected devices that brought down DNS services for large portions of the Internet [3]; and even the ability to take over physical systems like connected cars [4] with potentially dangerous implications for personal safety. These events have led both consumers and industry to re-evaluate the impact of such devices on their systems.

Beyond commercial and industrial concerns, US national security is increasingly dependent on the proper and secure operation of a myriad of platforms incorporating IoT sensors and actuators. This includes the protection and reliable operation of platforms and support systems making up National Security Systems (NSS). Almost 50% of the DoD purchases each year are for Commercial Off The Shelf (COTS) products including ID Cards, embedded medical devices, vehicles, security monitors, lights, and mobile phones [5]. The protection of NSS and support for operational missions is dependent on well-functioning public services, trustworthy commercial products,

Authorized licensed use limited to: Malta College of ArtsScience&Tech. Downloaded on October 23,2025 at 11:02:28 UTC from IEEE Xplore. Restrictions apply.

978-1-5386-7185-6/18/$31.00 ©2018 IEEE

and the ability to protect against misuse of data by malevolent actors.

These critical use cases motivate the need for a practical solution to runtime integrity verification for resource-constrained systems.

A. Threat Model

A large subset of adversarial behavior is intentional attacks on the system from the network. Defense against physical attacks is not a focus of this research. Important attacks are those which are remotely implanted and persist across updates, allowing an attacker to either deny service to the user, use the device to attack other systems, or access sensor data to leak information about the user. It is assumed that an attacker can exploit a flaw in the primary application to achieve these goals. Such exploits can include any traditional software attack vectors. Detection and containment of such access is a primary goal of this research. We assume that an adversary may be aware that there is an integrity agent on the platform.

B. Target Platform

This research focuses on runtime verification of 32-bit micro-controller (MCU) platforms, such as the ARMv7-M and ARMv8-M architecture. These platforms provide a high level of capability with a limited power budget, but lack a full Memory Managuement Unit (MMU) to provide virtual address isolation between tasks. More capable platforms based upon 32-bit or 64-bit CPUs (e.g. ARMv7-A, ARMv8-A, MIPS) are not considered, as they have a full MMU and can use existing runtime verification agents. Verification on less capable systems, such as 8-bit and 16-bit micro-controllers, remains an open challenge and is not considered in this research. Recent trends in commercial applications show a trend away from 8-/16-bit platforms and towards 32-bit MCUs [6].

C. Challenges

Resource-constrained devices present a number of challenges for runtime verification agents, including:

- **Lack of Hardware Trust**: While a few systems include basic features like signed boot [7], hardware cryptography [8], and secure SRAM [9], more robust trust mechanisms are not available.
- **Resource Constraints**: Devices contain only enough compute, memory, storage, and power necessary to perform their primary task. This leaves few to no resources left for a measurement agent.
- **Minimal Memory Separation**: Most micro-controllers lack a full Memory Management Unit (MMU) to provide task isolation, meaning that any compromised task can access the entire system.
- **Diversity**: Many current devices use custom firmware provided by the vendor, making it difficult to write a common agent that integrates with all devices. However, recent trends [6] indicate a move away from proprietary firmware and toward open source platforms such as FreeRTOS [10] and mbedOS [11].

TABLE I
DESIRED PROPERTIES OF TRUSTWORTHY M&A FRAMEWORKS

Desired Property	Where	Description
Completeness	Device	Collected evidence must provide a comprehensive view of the current state of the device
Freshness	Delegate	Evidence must be timely and current
Semantic Explicitness	Delegate	Evidence must be relevant to the current attestation need
Authenticity	Device	Evidence must be verified to come from the device under test
Correctness	Device	All components of the framework must have a sound design and be correctly implemented
Protection	Device	Components of the framework must be isolated
Flexibility	Delegate	The framework must be capable of supporting multiple deployment architectures
Usability	Delegate	Presentation of the evidence should be applicable to the given appraisal scenario

- **Communications**: Despite initiatives such as [12], [13], no widely-used standard for communication and managing devices has emerged. Therefore, an agent needs to support multiple diverse protocols over both wired and wireless media.

D. Measurement & Attestation

Measurement and attestation remains an active area for research on all types of platforms. While an exhaustive list of M&A-related research is outside the scope of this paper, we highlight some driving principles for IoTA from previous research below. See [14] for a more complete review of M&A systems and concepts.

Previous research [15], [16] has shown that well designed M&A systems must enforce a set of properties, summarized in Table I, in order to guarantee the trustworthiness of the resulting evaluation. These properties guide the design of IoTA, so that the collected evidence supports trustworthy appraisal evaluations in a wide variety of deployment scenarios. Other researchers have proposed similar, complimentary lists of properties (see e.g. [17]).

Maat [14] is an extensible, open M&A framework designed from the ground up to meet all the desired properties. It provides a solid foundation for IoTA to build upon due to its careful design and demonstrated success on conventional platforms. However, Maat relies on a complex negotiation scheme using contracts passed amongst peers. This allows Maat to be used in both rigid hierarchical deployment structures as well as more ad-hoc scenarios. It is designed for modern, multi-purpose operating systems with mandatory access controls and hardware-supported separation. Such a system cannot transfer as-is to resource-constrained platforms. However, the modular nature of Maat allows for extensions to incorporate non-traditional systems. IoTA's on-device runtime verification agent is designed to *optionally* integrate with Maat, allowing a single framework to verify all types of systems. However, IoTA is not dependent upon Maat, and can be deployed on its own or as component of other M&A platforms such as OpenAttestation [18] and OpenCIT [19].

Authorized licensed use limited to: Malta College of ArtsScience&Tech. Downloaded on October 23,2025 at 11:02:28 UTC from IEEE Xplore. Restrictions apply.

E. Related Work

Much existing integrity measurement research for resource-constrained platforms either relies on hardware modifications, is purely software-based, or is only applicable to specific scenarios. IoTA aims to move beyond these limitations while also integrating with existing M&A frameworks aimed at conventional platforms.

Vehicle systems are a rich area for integrity research. Oguma et al. [20] worked around the lack of trust on the vehicle's internal controllers by using the rigid tree structure of the system and pre-distributed keys to create a remote attestation system not unlike remote attestation using TPMs. Their system, however, relies on the rigidity of the network architecture and thus is not widely applicable.

For generic sensor data, YouProve [21] uses Android with a TPM emulator to monitor and sign sensor data. Liu et al. [22] describe a system similar to existing M&A architectures for collecting sensor data from edge devices that leverages virtualization for separation and security. Both of these systems require more capable devices (with MMUs) than those targeted in this research.

Another related research area is swarm attestation. Systems like [23], [24] lower the impact of collecting measurements from a large number of similar devices. Unfortunately, these methods are scenario-specific and not applicable to a wide variety of deployment scenarios.

Other researchers use novel hardware protection mechanisms that can be extended to typical microcontrollers. Examples include Intel TyTAN [25] and TrustLite [26]. These systems leverage prototype hardware to separate measurements into trusted threads called *trustlets*. SMART [27] and SANCUS [28] also rely on hardware modifications to a platform's MMU to ensure integrity. However, the modified hardware required for these systems is not currently available.

Other researchers [29], [30] attempt to side-step the lack of hardware-based trust mechanisms by proposing software-based schemes. These schemes rely on precise timing to show that the code itself has not been compromised because any new code introduced would change the timing of the measurement. These software-based systems have been shown to be more susceptible to exploitation than hardware-based systems [31], but remain a last resort in the absence of hardware support for trust.

III. Architecture

IoTA provides a minimal, on-device, trustworthy runtime agent which guarantees the properties of protection, completeness, authenticity, and correctness of the runtime evidence it gathers. In conjunction with more capable upstream devices, the combined system can guarantee all the properties of a well-designed M&A framework. Its on-device architecture is shown in Figure 1.

A. Delegating Properties

To support runtime verification in a resource-constrained environment, we examined all aspects of the framework for their purpose and implementation. Coker et al. [16] state that each individual device must implement support for all of the desired properties for maximum flexibility. This allows each device to operate independently with peer-to-peer negotiation for complex architectures. However, the negotiation process inherent in supporting all the properties requires long-term state and policy storage, as well as computational capabilities beyond those available on our target platforms.

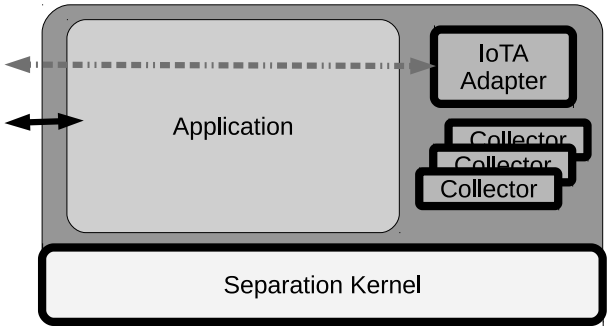

Fig. 1. IoTA architecture. Areas in thick black borders are isolated from the rest of the system.

Examining the properties in Table I, we observe that some properties specify *what* evidence to collect and *when* to collect it, while other properties specify *how* to collect and protect the evidence itself. Because the evidence is located on the device, properties that involve collection and protection of evidence *must* be implemented on the device itself. However, determining what and when to collect evidence is a negotiation, and we assert that another system with sufficient information about the device could perform that negotiation on the device's behalf. While this "split" architecture is unable to support every deployment scenario envisioned in [16], it provides a means for resource efficient integrity measurement in a wide range of deployment scenarios of interest to IoT.

A summary of the eight properties and whether they may be delegated is available in the "Where" column in Table I. In our architecture, the properties of completeness, authenticity, correctness, and protection all deal with *how* to gather and report evidence, while the remainder of the properties are candidates to be delegated off-device to a more capable system. This vastly simplifies the design and implementation of the on-device agent as it only has to guarantee four properties instead of all eight.

B. Device Components

IoTA limits the functionality needed on the device to the bare minimum necessary to guarantee the four on-device properties. Authenticity, correctness, and protection properties require a secure runtime on the device which cannot be modified by unauthorized applications. A secure evidence collection architecture is is straightforward once a protected area is available. This protected area may be provided by software hypervisors, separation kernels, or hardware partition schemes. While full secure partition schemes like ARM TrustZone [32] or Intel SGX [33] are not available on IoTA's

Authorized licensed use limited to: Malta College of ArtsScience&Tech. Downloaded on October 23,2025 at 11:02:28 UTC from IEEE Xplore. Restrictions apply.

target platforms, software hypervisors can provide isolation by leveraging a micro-controller's existing, if limited, memory protection support. Future IoTA platforms can leverage ARM's recently-announced Platform Security Architecture (PSA) [34] to provide hardware separation. Private keys and evidence collectors are stored in the protected areas and are only accessible through a defined API. Evidence is collected and cryptographically signed with a nonce before leaving unprotected memory for evaluation. Strong separation ensures that evidence is authentic, correct, and protected from undetectable modification.

Completeness is a function of the set of evidence collectors available on the device and is bounded only by available resources. Our architecture calls for multiple evidence collectors on the device, which can be selectively used for different attestation scenarios. Examples of evidence collectors range from simple measurements like hashes of flash-based storage and requesting device-specific identity certificates, up to advanced collectors that perform memory introspection of the primary application (similar to LKIM [15]). Where possible, measurements should include evidence that the separation kernel is correctly enforcing the separation necessary to ensure the integrity of the software running in the protected area.

IV. Prototype

With the prior architecture in mind, we developed a prototype of IoTA to explore its practical implementation. We chose to prototype IoTA using readily available off-the-shelf hardware and software focused on IoT operating environments. The primary goals of this prototype are threefold: (1) to explore how difficult it is to implement our architecture, (2) to build practical runtime evidence collectors to demonstrate the effectiveness of the architecture in detecting real-world corruption in example scenarios, and (3) to quantify the overhead associated with the on-device agent. Secondary goals include providing a foundation for further runtime integrity research, and examining the portability of the architecture to multiple target platforms.

Note that this prototype is a proof of concept for the basic architecture and capabilities of IoTA, and as such has limited optimizations with respect to code size and runtime efficiency. Future improvements are discussed in Section V.

A. Choosing A Representative Platform

We selected ARM's mbedOS [11] and uVisor [35] software platform and the Freescale FRDM-K64F development board as the hardware platform. MbedOS and uVisor are ideal candidates because they combine both a real-time operating system (RTOS) with a "micro" hypervisor, which provides some separation (via *boxes*) using the Cortex-M's hardware Memory Protection Unit (MPU). The FRDM-K64F is the primary supported hardware platform for mbedOS and uVisor development. While these platforms are used for our prototype, IoTA is designed to avoid platform-specific features where possible, and is easily portable to other RTOSs and platform separation mechanisms.

B. Evidence Request/Response Protocol

The IoTA prototype implements a simple binary request/response protocol with cryptographic signing and mutual authentication. Both the requester and responder check all messages for authenticity. Upon receiving a verified request, the IoTA agent launches the requested evidence collectors, waits for the evidence to return, and signs and sends the response.

Messages are transferred over a TLS-encrypted TCP/IP connection with mutual certificate checking to prevent man-in-the-middle attacks. The certificates used for the TLS tunnel do not need to be the same certificates used to authenticate requests and responses. TLS-based communication provides an implicit session based on the connection. However, it also adds significant overhead in both power, storage size, and latency, as seen in Section IV-E, and may not be practical. The IoTA protocol itself is transport-agnostic, so lighter-weight session-less protocols like CoAP [36] or MQTT [37] may be more suitable.

C. Runtime Evidence Collectors

At an abstract level, collectors are evidence primitives which populate *nodes* in a larger evidence *graph* that provides a complete and relevant picture of the current state of the device. This evidence graph is evaluated to determine the integrity of the system under test. The set of collectors must be complete and relevant to the widest range of desirable integrity scenarios. The types of collectors we implemented here are in part analogues to the collectors necessary on a traditional platform, and adapted for resource-constrained platforms.

Collectors are self-contained execution units with a simple API for the IoTA agent to invoke to collect evidence. Collectors may be isolated in their own uVisor box to grant the collector, and only the collector, the privileges needed to take a measurement.

Evidence produced by a collector must be comprehensible and verifiable by the external integrity evaluator. As such, it must measure some state that is expected to be known ahead of time and invariant, or exist within a small, defined set of expected states. This means that collectors are closely tied to the specific firmware and application layout of the system. It is envisioned that a device's developer will identify such invariants and build collectors specific to their application. Because of this close coupling, it is difficult, or impossible, to build generic collectors that work on all devices. However, the IoTA agent component is generic. So implementers who use IoTA can focus on building only the platform-specific collectors and re-use the IoTA agent for processing and dispatching integrity requests.

We implemented the following collectors in our prototype:

- **Static Hash of Persistent Storage**: Verify contents of flash memory using SHA-256 to detect persistent modifications.
- **Runtime analysis of the Interrupt Vector Table**: Verify that the device's interrupts are properly routed.

Authorized licensed use limited to: Malta College of ArtsScience&Tech. Downloaded on October 23,2025 at 11:02:28 UTC from IEEE Xplore. Restrictions apply.

TABLE II
IoTA PROTOTYPE CODE-SIZE OVERHEAD

	With uVisor (KB)	Without uVisor (KB)
mbedOS/uVisor	126	124
mbedTLS	102	102
Example App	24	8
IoTA Adapter and Collectors	43	10
Full System	295	244

TABLE III
ARCHITECTURE TIMING LATENCIES

	Latency (s)
TLS Handshake	2.150
TLS Recv	0.035
IoTA Verify	0.002
Measurement	See Table IV
IoTA Response	0.025
TLS Send	0.040
Total	2.252 + time(Measurement)

- **Runtime Memory Introspection of uVisor**: Verify the MPU configuration, the number of threads, and other critical dynamic uVisor content is as expected.
- **Runtime Application Introspection**: Verify application critical state in memory is configured as expected.

This set of collectors is sufficient to verify a large portion of the ciritical state of our prototype device, and provides a rich testing ground to test our implementation. They should also serve as examples for other devices to implement their own relevant set of collectors.

D. Evaluation

We developed a simple example TCP/IP-based application that broadcasts readings from the on-device accelerometer over the network once every 5 seconds, while also accepting requests over the network for individual readings. Though simplistic, this type of application exercises many aspects of the system, combining network processing, user input, and working with on-device sensors.

Using the collectors mentioned above, we were able to verify the integrity of the running system with minimal impact on the primary application. Additionally, we are able to demonstrate that when malicious input is provided to the application which modifies its data structures, the application memory introspection collector reflected this unexpected state, and integrity evaluation fails. This capability was demonstrated using multiple K64F development platforms IoTA and integrated with the Maat M&A framework [38].

E. Overhead Metrics

Next we quantify IoTA's impact on the size, latency, and power of our target application platform, while noting our prototype is a proof of concept with many potential opportunities for optimization.

Table III summarizes the overall on-device latency associated with processing, collecting, and returning evidence. The latency is divided into 5 stages: (1) Establishing the TLS tunnel, (2) receiving the request from the socket, (3) verifying the request, (4) dispatching and collecting the evidence, (4) generating and signing the response, and (5) sending the response over the TLS tunnel. Of particular note is that a significant portion is dedicated to establishing the TLS tunnel for secure communication. All other latencies are relatively small. Table IV shows the latencies associated with individual collectors. Note that the *NULL Collector* returns immediately providing a way to isolate IoTA overhead from the collector used. Most collectors complete in negligible time except for hashing our example application, which is directly proportional to the chosen hash algorithm and application size.

TABLE IV
COLLECTOR-SPECIFIC TARGETS AND LATENCIES

Type	Target	Latency(μs)
NULL Collector	N/A	0.89
Runtime Memory Introspection	Application Data Structures	6.70
Runtime Memory Introspection	uVisor Data Structures	62.20
Runtime Hash	uVisor Vector Table	301.00
Static Hash	uVisor Code	6100.00
Static Hash	Application Code	127800.00

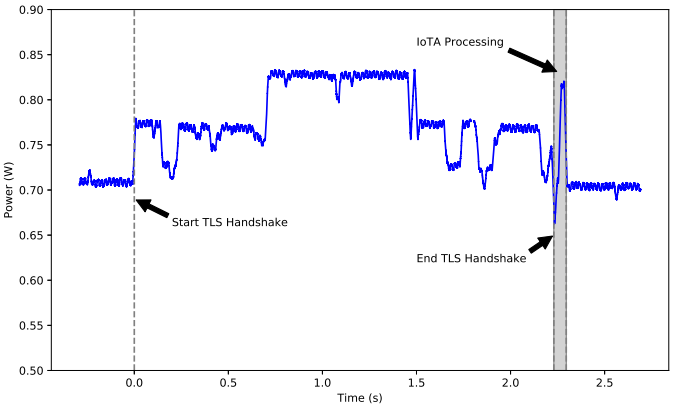

Fig. 2. Platform/IoTA Power Overhead With Annotations.

Power overhead is shown in Figure 2. The peak during IoTA agent processing is due to the signing and sending the measurement, not the measurement itself. Empirical instantaneous power overhead for the IoTA prototype is approximately 85mW over 2.3s, depending on the requested measurement type, with the vast majority of power and time comsumed by the TLS handshake.

V. CONCLUSIONS AND FUTURE WORK

Currently we are improving our prototype in preparation for an open source release, including porting the system to more platforms, comparing the resulting implementations, and developing more example collectors. We are also evaluating other lightweight cryptography and communication systems which may be better suited for resource-constrained devices. We will also support new hardware separation mechanisms as they become available, such as ARM's PSA.

Authorized licensed use limited to: Malta College of ArtsScience&Tech. Downloaded on October 23,2025 at 11:02:28 UTC from IEEE Xplore. Restrictions apply.

IoTA is a generic architecture for open, trustworthy, agent-based runtime integrity collection and verification on resource-constrained devices. IoTA preserves the desirable properties — *completeness, authenticity, correctness,* and *protection* — required by trustworthy integrity evaluation frameworks. Our novel split architecture achieves the remaining properties by delegating heavyweight negotiation components to more capable systems on behalf of the resource-constrained device. We have demonstrated that tight coupling between the end device and the gateway does not alter the usability of the architecture. We have also presented a working, practical implementation of this architecture; demonstrating that agent-based runtime measurement is capable of detecting advanced runtime memory corruption on resource-constrained devices with manageable overhead. We hope our implementaion spurs further research into trustworthy runtime integrity for resource-constrained platforms.

References

[1] S. Smith, *The Internet of Risky Things*. O'Reilly Media, 2017.

[2] B. Krebs, "The target breach, by the numbers," *Krebs on Security*, vol. 6, 2014.

[3] ——, "DDoS on Dyn impacts Twitter, Spotify, Reddit," *Krebs On Security (October 16, 2016)*, vol. 20, no. 16, p. 1, 2016.

[4] T. Ring, "Connected cars the next target for hackers," *Network Security*, vol. 2015, no. 11, pp. 11–16, Nov. 2015. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1353485815301008

[5] Trusted Systems Research Group, DoD, "Internet of Things Strategy," 2016, unpublished Internal Document.

[6] 2017 embedded market survey | embedded. [Online]. Available: https://www.embedded.com/electronics-blogs/embedded-market-surveys/4458724/2017-Embedded-Market-Survey

[7] Securing the IoT: Part 2 - secure boot as root of trust. [Online]. Available: https://www.embedded.com/design/safety-and-security/4438300/Securing-the-IoT--Part-2---Secure-boot-as-root-of-trust-

[8] S. Ravi, A. Raghunathan, P. Kocher, and S. Hattangady, "Security in embedded systems: Design challenges," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 3, no. 3, pp. 461–491, 2004.

[9] H. Handschuh, "Hardware-anchored security based on sram pufs, part 1," *IEEE Security & Privacy*, vol. 10, no. 3, pp. 80–83, 2012.

[10] R. Goyette, "An analysis and description of the inner workings of the freertos kernel," *Carleton University*, vol. 5, 2007.

[11] ARM, "ARM mBed," 2017. [Online]. Available: http://www.mbed.com/en

[12] AllSeen Alliance, "Framework AllSeen Alliance," 2017. [Online]. Available: https://allseenalliance.org/framework

[13] Samsung, "IoTivity," 2017. [Online]. Available: https://www.iotivity.org/

[14] J. A. Pendergrass, S. Helble, J. Clemens, and P. Loscocco, "Maat: A platform service for measurement and attestation." [Online]. Available: http://arxiv.org/abs/1709.10147

[15] P. A. Loscocco, P. W. Wilson, J. A. Pendergrass, and C. D. McDonell, "Linux kernel integrity measurement using contextual inspection," in *Proceedings of the 2007 ACM workshop on Scalable Trusted Computing*. ACM, 2007, pp. 21–29.

[16] G. Coker, J. Guttman, P. Loscocco, J. Sheehy, and B. Sniffen, "Attestation: Evidence and Trust," in *Information and Communications Security*, ser. Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2008, vol. 5308, pp. 1–18.

[17] G. Hunt, G. Letey, and E. Nightingale, "The seven properties of highly secure devices," Tech. Rep., March 2017. [Online]. Available: https://www.microsoft.com/en-us/research/publication/seven-properties-highly-secure-devices/

[18] Intel, "OpenAttestation/OpenAttestation," 2017. [Online]. Available: https://github.com/OpenAttestation/OpenAttestation

[19] Open cloud integrity technology (open CIT). [Online]. Available: https://01.org/opencit

[20] H. Oguma, A. Yoshioka, M. Nishikawa, R. Shigetomi, A. Otsuka, and H. Imai, "New Attestation Based Security Architecture for In-Vehicle Communication," in *IEEE GLOBECOM 2008 - 2008 IEEE Global Telecommunications Conference*, Nov. 2008, pp. 1–6.

[21] P. Gilbert, J. Jung, K. Lee, H. Qin, D. Sharkey, A. Sheth, and L. P. Cox, "YouProve: Authenticity and fidelity in mobile sensing," in *Proceedings of the 9th ACM Conference on Embedded Networked Sensor Systems*, ser. SenSys '11. ACM, pp. 176–189.

[22] H. Liu, S. Saroiu, A. Wolman, and H. Raj, "Software abstractions for trusted sensors," in *Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys '12. ACM, pp. 365–378.

[23] N. Asokan, F. Brasser, A. Ibrahim, A.-R. Sadeghi, M. Schunter, G. Tsudik, and C. Wachsmann, "SEDA: Scalable Embedded Device Attestation," in *Proceedings of the 22Nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '15. New York, NY, USA: ACM, 2015, pp. 964–975.

[24] A. Ibrahim, A.-R. Sadeghi, G. Tsudik, and S. Zeitouni, "DARPA: Device attestation resilient to physical attacks," in *Proceedings of the 9th ACM Conference on Security & Privacy in Wireless and Mobile Networks*. ACM, 2016, pp. 171–182.

[25] F. Brasser, B. El Mahjoub, A.-R. Sadeghi, C. Wachsmann, and P. Koeberl, "TyTAN: Tiny Trust Anchor for Tiny Devices," in *Proceedings of the 52Nd Annual Design Automation Conference*, ser. DAC '15. New York, NY, USA: ACM, 2015, pp. 34:1–34:6.

[26] P. Koeberl, S. Schulz, A.-R. Sadeghi, and V. Varadharajan, "TrustLite: A Security Architecture for Tiny Embedded Devices," in *Proceedings of the Ninth European Conference on Computer Systems*, ser. EuroSys '14. New York, NY, USA: ACM, 2014, pp. 10:1–10:14.

[27] K. Eldefrawy, G. Tsudik, A. Francillon, and D. Perito, "SMART: Secure and Minimal Architecture for (Establishing Dynamic) Root of Trust," in *NDSS*, vol. 12, 2012, pp. 1–15.

[28] J. Noorman, P. Agten, W. Daniels, R. Strackx, A. Van Herrewege, C. Huygens, B. Preneel, I. Verbauwhede, and F. Piessens, "Sancus: Low-cost Trustworthy Extensible Networked Devices with a Zero-software Trusted Computing Base," in *Proceedings of the 22Nd USENIX Conference on Security*, ser. SEC'13. Berkeley, CA, USA: USENIX Association, 2013, pp. 479–494.

[29] A. Seshadri, M. Luk, E. Shi, A. Perrig, L. van Doorn, and P. Khosla, "Pioneer: Verifying Code Integrity and Enforcing Untampered Code Execution on Legacy Systems," in *Proceedings of the Twentieth ACM Symposium on Operating Systems Principles*, ser. SOSP '05. New York, NY, USA: ACM, 2005, pp. 1–16.

[30] X. Kovah, C. Kallenberg, C. Weathers, A. Herzog, M. Albin, and J. Butterworth, "New Results for Timing-Based Attestation," in *2012 IEEE Symposium on Security and Privacy (SP)*, May 2012, pp. 239–253.

[31] U. Shankar, M. Chew, and J. D. Tygar, "Side Effects Are Not Sufficient to Authenticate Software," in *Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13*, ser. SSYM'04. Berkeley, CA, USA: USENIX Association, 2004, pp. 7–7.

[32] A. Rane, "Securing the edge with ARM TrustZone for v8-M," Mar. 2017. [Online]. Available: http://embedded-computing.com/articles/securing-the-edge-with-arm-trustzone-for-v8-m/

[33] I. Anati, S. Gueron, S. Johnson, and V. Scarlata, "Innovative technology for cpu based attestation and sealing," in *Proceedings of the 2nd international workshop on hardware and architectural support for security and privacy*, vol. 13, 2013.

[34] A. Ltd. Platform security architecture. [Online]. Available: https://developer.arm.com/products/architecture/platform-security-architecture

[35] "uvisor: mbed OS uVisor – device security layer for ARMv7m microcontrollers," original-date: 2014-12-11T11:59:50Z. [Online]. Available: https://github.com/ARMmbed/uvisor

[36] Z. Shelby, K. Hartke, and C. Bormann, "The Constrained Application Protocol (CoAP)," RFC 7252 (Proposed Standard), Internet Engineering Task Force, Jun. 2014, updated by RFC 7959. [Online]. Available: http://www.ietf.org/rfc/rfc7252.txt

[37] "Information technology – Message Queuing Telemetry Transport (MQTT) v3.1.1," International Organization for Standardization, Geneva, CH, Standard, Jun. 2016.

[38] J. Clemens, R. P. DoD, and B. Sherrell, "Iota: Iot assurance (host 2017 hardware demo submission)," 2017, demo.

Authorized licensed use limited to: Malta College of ArtsScience&Tech. Downloaded on October 23,2025 at 11:02:28 UTC from IEEE Xplore. Restrictions apply.